第8章　数组

目 录

1 数组的定义与引用

2 字符数组

3 数组与函数

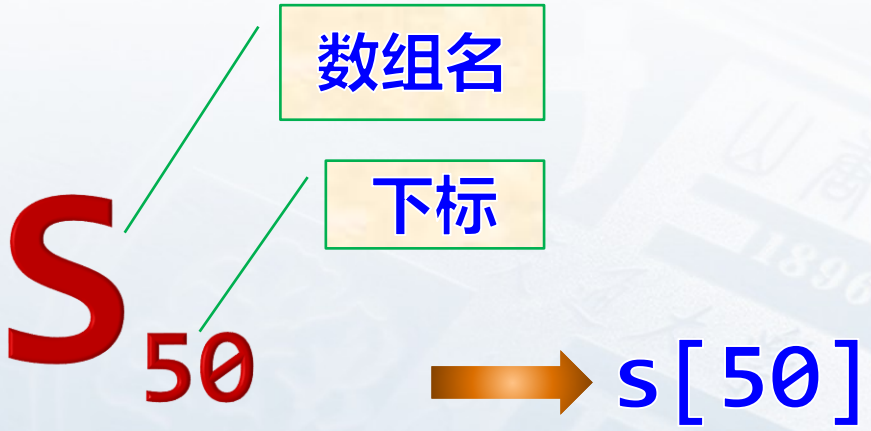
4 二维数组

学习目标

1. 掌握一维数组、二维数组的定义及数组元素的初始化和引用
2. 理解数组在内存中的存储格式
3. 掌握字符数组的定义和初始化及常用字符串处理函数的使用
4. 掌握数组和数组元素作为函数参数的使用方法
5. 掌握常用的排序和查找方法
6. 掌握x86平台和鲲鹏平台程序设计差异

为什么需要数组

向计算机输入全班50个学生一门课程的成绩，并排序



解决方法

用50个float型简单变量表示学生的成绩



- 烦琐，如果有1000名学生怎么办呢?
- 没有反映出这些数据间的内在联系，实际上这些数据是同一个班级、同一门课程的成绩，它们具有相同的属性。

数组

- ★ 数组是一组有序数据的集合。数组中各数据的排列是有一定规律的，下标代表数据在数组中的序号。
- ★ 用数组名和下标即可唯一地确定数组中的元素。
- ★ 数组中的每一个元素都属于同一个数据类型。

定义数组

怎样定义数组

怎样使用数组

定义一维数组

类型说明符 数组名[常量表达式];

```
int a[10];
```

整型数组，即数组中的元素均为整型

数组名为a

数组包含10个整型元素

① 数组名的命名规则和变量名相同，遵循标识符命名规则。

② 在定义数组时，需要指定数组中元素的个数，方括号中的常量表达式用来表示元素的个数，即数组长度。

③ 常量表达式中可以包括常量和符号常量，不能包含变量。

④ 数组和变量一样，必须先定义，后使用。

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

相当于定义了10个简单的整型变量

强调一下 Emphasize

📖数组元素的下标从0开始，用 `int a[10];` 定义数组，则最大下标值为9，不存在数组元素a[10]。

定义一维数组

一维数组的顺序存储 数组元素在内存中顺序存放，数组元素在内存中是从低地址开始顺序存放的，各元素的存储单元占用内存大小相同，它们的地址是连续的。各元素的存储单元之间没有空隙，可以从数组第一个元素存储单元的起始地址计算出任意一个元素存储单元的起始地址。

```
int array[5];
```

//数组名字array是数组首元素的内存地址

数组元素

array[0]	0x4C40
array[1]	0x4C44
array[2]	0x4C48
array[3]	0x4C4C
array[4]	0x4C50

16进制
内存地址

引用一维数组元素

数组名[下标]

强调一下 Emphasize

- 只能引用数组元素而不能一次性地整体调用整个数组全部元素的值。
- 数组元素与一个同类型简单变量的地位和作用相似。
- 下标可以是整型常量或整型表达式。

定义数组时用到的数组名[常量表达式]和引用数组元素时用的数组名[下标]形式相同，但含义不同。

```
int a[10];
//定义数组，指定数组包含10个元素，每个均为int
t=a[6];
//a[6]表示引用a数组中序号为6的元素
```



数组的下标都有一个范围，即“0~[数组长度-1]” 假设数组的长度为10，其下标范围为0～9。当访问数组中的元素时，下标不能超出这个范围，否则程序会报错。

单选题 100分

设置

以下关于数组的说法，错误的是(　　)

A 数组用数组名来标识。一个数组名用来表示一组同类型的数据，这批同类型的数据被称为数组元素或分量。

B 数组描述的是顺序数据结构，即数组元素在逻辑上相邻，在物理也相邻，即在内存中它们是连续存放的。

C 数组是具有一定顺序关系的若干相同或不同类型的数据集合体，数组属于复合类型。

D 数组和变量一样，必须先定义，后使用。

提交

多选题 100分

设置

```
#define N 10
int a[N];
```

以下哪些操作是正确的(　)

A　a[0]=10;

B　a[10]=100;

C　for(i=0;i<N;i++)　scanf("%d",&a[i]);

D　for(i=0;i<=10;i++)　printf("%d ",a[i]);



在for循环中使用scanf("%d ", &a[i])时，输入回车是不能结束的。因为scanf函数在读取输入时，会跳过前面的空白字符，直到遇到非空白字符。

空格、制表符和换行符

在scanf的格式字符串中包含一个空格(如"%d ")，程序会等待直到用户输入一个数字后跟一个非空白字符才能结束输入。若用户只输入数字后按回车，由于回车本身是一个换行符，scanf会继续等待直到下一个非空白字符出现。

```
int i, a[N];
for (i = 0; i < N; i++)
```

不要加空格

提交

引用一维数组元素

示例 对10个数组元素依次赋值为0,1,2,3,4,5,6,7,8,9，要求按逆序输出。

```
#include <stdio.h>
int main(void)
{
    int i, a[10];
    for (i = 0; i <= 9; i++) // 对数组元素a[0]~a[9]赋值
        a[i] = i;
    for (i = 9; i >= 0; i--) // 输出a[9]~a[0]共10个数组元素
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

📖 第1个for循环使a[0]～a[9]的值为0～9

a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
0	1	2	3	4	5	6	7	8	9



📖 第2个for循环按a[9]~a[0]的顺序输出各元素的值

单选题 100分

设置

以下关于数组的说法，正确的是(　　)

A 数组定义中，表达式的值可以是符号常量、变量等。如

```
int n=3,num[n*3];
```

B 数组的下标是用圆括号括起来的，如

```
int score[5];   score(2)=92;
```

C 不但可以逐个引用数组元素，还可以整体引用数组。如

```
int a[5];  a={10,20,30,40,50};
```

D 数组的命名规则同于变量的命名规则。如 `int i,j,a[10];`

强调一下 Emphasize

★ 支持C99标准的编译器，定义数组时可以用变量(但已经有确定值)指定数组长度。

提交

一维数组的初始化

在定义数组的同时给各数组元素赋值，称为数组的初始化。

1．在定义数组时对全部数组元素赋予初值。

```
int a[10]={0,1,2,3,4,5,6,7,8,9};
```

- 将数组各元素的初值顺序放在一对花括号内，数据间用逗号分隔。花括号内的数据就称为初始化列表。

2．可以只给数组中的一部分元素赋值。

```
int a[10]={0,1,2,3,4};
```

- 定义a数组有10个元素，但花括号内只提供5个初值，表示只给前面5个元素赋初值，系统自动给后5个元素赋初值为0。

3．给数组中全部元素赋初值为0。

```
int a[10]={0,0,0,0,0,0,0,0,0, 0};
```

或

```
int a[10]={0};  //未赋值的部分元素自动设定为0
```

4．在对全部数组元素赋初值时，由于数据的个数已经确定，可以不指定数组长度。

```
int a[5]={1,2,3,4,5};
```

或

```
int a[ ]={1,2,3,4,5};
```

强调一下 Emphasize

★ 如果数组长度与提供初值的个数不相同，方括号中的数组长度不能省略。

一维数组的初始化_总结

一维数组的初始化　数组初始化常见的方式有三种：

第1种	直接对数组中的所有元素赋值 int a[5]={1,2,3,4,5};
第2种	只对数组中的一部分(至少1个)元素赋值 int a[5]={1,2,3};
第3种	对数组全部元素赋值可不指定长度 int a[]={1,2,3,4,5};

未赋值元素将自动赋值为0

数组长度为5

敲黑板！

- 如果没有赋任何一个初值，C/C++不能自动初始化数组元素，数组元素将包含垃圾数据。
- 声明数组时，可在前面加上关键字static，如：`static int a[5];`
  - 若未对静态数组进行初始化，则数组的所有元素将自动获取初始值0。静态数组的初始化是在编译时完成的，以后不用再初始化，而是直接使用原有值。

单选题 100分

设置

语句 `int a[]={0,1,2};` 执行后，数组的元素个数是(　　)。

A　2

B　3

C　4

D　不知道

例6-1

提交

一维数组数据的输入

西南交通大学 Southwest Jiaotong University

- ***不能对数组整体操作***，只能对数组元素***逐个***操作；
- 通过***循环控制下标变量***访问数组中每个元素(***遍历***)

通用

a[i]	a[0]	2
	a[1]	4
	a[2]	6
	a[3]	8
	a[4]	10

```
int i, a[5];
for (i = 0; i < 5; i++)
    scanf("%d", &a[i]);
```

```
#define N 5
int i, a[N];
for (i = 0; i < N; i++)
    scanf("%d", &a[i]);
```

一改全改

```
int a[50],n,i;
printf("请输入个数:");
scanf("%d",&n);
for (i=0;i<n;i++)
    scanf("%d",&a[i];
```

- **数组元素个数未知的解决方案一**
  1. 定义大数组，如50；
  2. 输入具体个数n(<=50)
  3. 用n控制数组元素(***有效长度***)
- **方案二**：动态存储分配(见指针)

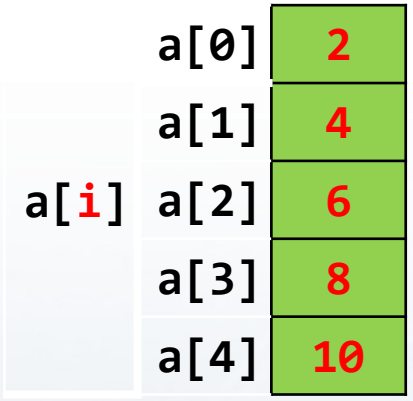


例6-2

例6-3

一维数组数据的输出

通过 ***下标变量遍历*** 数组中每个元素




```
#define N 5
int i, a[N];
for (i = 0; i < N; i++)
    scanf("%d", &a[i]);
```

```
// 正序输出
for (i = 0; i < N; i++)
    printf("%d ", a[i]);
```

```
// 逆序输出
for (i = N - 1; i >= 0; i--)
    printf("%d ", a[i]);
```

```
// 输出满足条件的数据
for (i = N - 1; i >= 0; i--)
   if(条件) printf("%d ", a[i]);
```

一维数组的操作

- 通过 ***下标变量遍历*** 数组中每个元素
- 计算（累和、均值、查找、最大最小等）

```
for(i=0;i<N;i++)
```

计算功能

```
s=s+a[i];
```
循环外：`m=s/N;`

求累和

顺序查找x

```
if(a[i]==x) printf("%d",i);
```

```
if(a[i]%2==0) c++;
```

统计满足条件的数

找出最大/最小值

找出最小值及下标

```
m=a[0];
for (i=1;i<N;i++)
    if(a[i]<m) m=a[i];
printf("%d",m);
```

```
if(a[i]<m) {m=a[i]; p=i;}
```

多选题 100分

设置

以下说明正确的是(　　)

A　C中各种数据类型的变量在定义后会被自动初始化为0值。

B　定义数组时可以用变量来定义数组的大小，但不能用表达式。

C　定义一个数组后该数组的最小下标取值是0。

D　数组是C的基本数据类型。

E　同一数组的所有数组元素在内存中是连续存放的。

F　在C中数组是具有一定顺序关系的若干相同类型变量的集合体。

提交

经典示例 用数组来处理求Fibonacci数列问题。

```
#include <stdio.h>
int main(void)
{
    int i;
    int f[20] = {1, 1}; // 对最前面两个元素f[0]和f[1]赋初值1
    for (i = 2; i < 20; i++)
        f[i] = f[i - 2] + f[i - 1]; // 先后求出f[2]~f[19]的值
    for (i = 0; i < 20; i++)
    {
        if (i % 5 == 0)
            printf("\n");      // 控制每输出5个数后换行
        printf("%12d", f[i]); // 输出一个数
    }
    printf("\n");
    return 0;
}
```

例6-4

例6-5

```
           1           1           2           3           5
           8          13          21          34          55
          89         144         233         377         610
         987        1597        2584        4181        6765
```

一维数组程序应用举例

例6-6 某班N名同学以匿名方式对班长的工作进行满意度评价，给出的分值为1到10，1分表示非常不满意，10分表示非常满意。统计出调查的结果。

- **思路分析**：明确要解决的问题是统计学生给出的各个分数的数目，针对问题，进行数据结构设计，需要引入两个数组
  - 一个整型数组score有N个元素，存放学生打出的分数，
  - 另一个整型数组result有11个元素，用于存放学生可能打出的10种分数的个数。我们不用result[0]这个元素，为的是将1~10分数值恰好当作下标来访问result数组，这样有助于提高程序的清晰度。

注意

对统计各分数数目的循环，只用了语句：`++result[score[i]];`

根据score[i]的值(只可能是1到10)，对数组result中相应的元素进行增1处理。假设元素score[30]值为6，说明第31个同学给班长投了6分，那6分对应的结果元素result[6]就应该加1。

插入问题

示例 在一个数组中某位置插入指定数据



- 数组定义多一个空间；
- 确定欲插入位置下标p；
- 向后移动；
- 插入

将某个数插入到a6后面

a1	a2	a3	a4	a5	a6	a7	a8	a9	

后移时i变化：n-1~p+1，i--

i--

i=p　　i-1　i=n-1

a1	a2	a3	a4	a5	a6		a7	a8	a9

```
for(i=n-1;i>p;i--)
    a[i]=a[i-1]; //后移
a[p]=x;              //或a[i]=x;
```

★后移：a[i]=a[i-1];

★插入数据：a[p]=x;



插入问题

★ 把一个数插入到一组升序排列的数据中，并要求仍然保持升序。

★ 随机产生n个数(0~100)，找出最小数及下标，并在最小数后插入0。




```
#include <stdio.h>
#include <time.h>
#include <stdlib.h>
#define N 10
int main(void)
{
    int a[N], i, min, n, p = 0;
    printf("产生的%d个0~100之间的随机数为: "
    srand(unsigned(time(0)));//初始化随机数
    for (i = 0; i < N - 1; i++)
    {
        a[i] = rand() % 101;//生成随机数
        printf("%3d", a[i]);
    }
```

```
    min = a[0];
    for (i = 1; i < N - 1; i++) // 确定最小数及下标
        if (a[i] < min)
        {
            min = a[i];
            p = i;
        }
    printf("\n最小数为: %d, 下标为: %d\n", min, p);
    for (i = N - 1; i >= p; i--) // 后移:下标n-1~p+1
        a[i + 1] = a[i];
    a[p + 1] = 0;//插入数据到指定位置
    printf("插入后数组元素为: ");
    for (i = 0; i < N; i++)
        printf("%3d", a[i]);
    return 0;
}
```

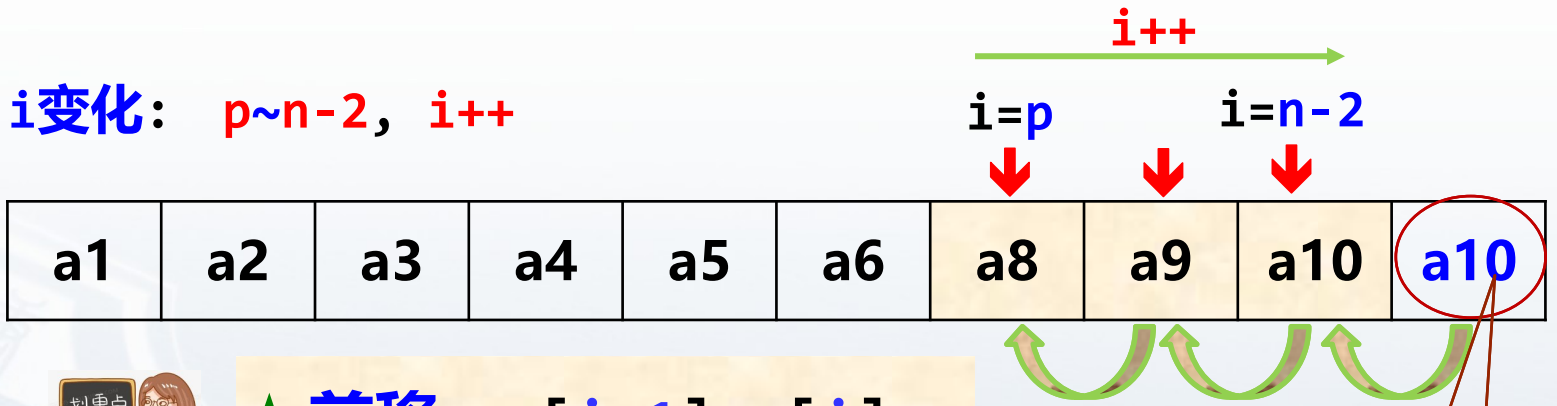
删除问题

示例 删除一个数组中某位置数据



- 确定欲删除位置下标p;
- 向前移动(覆盖);
- 有效数组长度减少1

i变化: p~n-2, i++




```
//删除a[p]
for(i=p;i<n-1;i++)
    a[i]=a[i+1];     //前移
```



★前移: a[i+1]=a[i];

★有效数组长度: n--;

```
//输出
for(i=0;i<      ;i++)
    printf("%3d", a[i]);
```

☎ 删除一个数据后n--; 则填n

☎ 删除一个数据后无n--; 则填n-1

☎ 删除若干个数据后c++; 则填n-c

删除问题

示例 删除一个数组中指定数据

```
printf("\n请输入要删除的数据: ");
scanf("%d", &x);
for (i = 0; i < N; i++)
    if (ss[i] == x) // 确定欲删除数据的下标
    {
        p = i;
        flag = 1;
        break;
    }
```

敲黑板!

★ 确定删除位置

★ 前移删除

```
if (flag == 0)
{
    printf("\n无此数! ");
    exit(0);
}
else if (p == N - 1)
    n--; // 删除的是最后一个数
else
{
    for (i = p; i < N - 1; i++)
        ss[i] = ss[i + 1];
    n--; // 有效数据减1
}
printf("\n数组删除指定数据后为: ");
for (i = 0; i < N - 1; i++)
    printf("%d ", ss[i]);
```



★ 输入n个数，求出平均值、删除其中所有高于平均值的元素并输出。

★ 删除数组中的重复数据

字符数组

什么是字符数组

怎样使用字符串处理函数

定义字符数组

- 用来存放字符数据的数组是**字符数组**。字符数组中**一个元素**占内存**一个字节**。

```
char c[10];
c[0]='I'; c[1]=' ';c[2]='a';c[3]='m';c[4]=' ';c[5]='h';c[6]='a';c[7]='p';c[8]='p';c[9]='y';
```

c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]	c[8]	c[9]
I	␣	a	m	␣	h	a	p	p	y

- 由于**字符型数据**是以整数形式(**ASCII码**)存放的，因此也可以用**整型数组**来存放字符数据。

```
int c[10];
c[0]='a'; //合法，但浪费存储空间
```

字符数组的初始化

字符数组初始化，采用初始化列表，把各个字符依次赋给数组中各元素。

```
char c[10]={'I',' ','a','m',' ','h','a','p','p','y'};//把10个字符依次赋给c[0]~c[9]这10个元素
```

- 若在定义字符数组时不进行初始化，则数组中各元素的值是不可预料的。
- 若花括号中提供的初值个数(即字符个数)大于数组长度，则出现语法错误。
- 若初值个数小于数组长度，只将这些字符赋给数组中前面那些元素，其余元素自动定为空字符(即'\0')。

```
char c[10]={'c',' ','p','r','o','g','r','a','m'};
```

c[0]	c[1]	c[2]	c[3]	c[4]	c[5]	c[6]	c[7]	c[8]	c[9]
c	␣	p	r	o	g	r	a	m	\0

- 若提供的初值个数与预定的数组长度相同，定义时可以省略数组长度，系统会自动根据初值个数确定数组长度。

```
char c[]={'I',' ','a','m',' ','h','a','p','p','y'};      //数组c的长度自动定为10
```

- 定义和初始化一个二维字符数组。

```
char diamond[5][5]={{' ',' ','*'},{' ','*',' ','*'},{'*',' ',' ',' ','*'},{' ','*',' ','*'},{' ',' ','*'}};
```

引用字符数组中的元素

例6-7 合并两个已知的字符数组中的内容。

```
#include <stdio.h>
int main(void)
{
    char str1[4] = {'H', 'a', 'r', 'd'}; /*初始化字符数组str1*/
    char str2[4] = {'W', 'o', 'r', 'k'}; /*初始化字符数组str2*/
    char str3[8];                        /*定义字符数组str3, 接收合并结果*/
    int i;
    for (i = 0; i < 4; i++) /*将数组str1中的各个字符存放到数组str3中*/
        str3[i] = str1[i];
    for (i = 0; i < 4; i++) /*接着将数组str2中的各个字符存放到数组str3中*/
        str3[4 + i] = str2[i];
    for (i = 0; i < 8; i++)
        printf("%c", str3[i]);
    printf("\n");
    printf("\n");
    return 0;
}
```

字符串和字符串结束标志

- C语言中，将字符串作为字符数组来处理的。如“China”，存储为：

C	h	i	n	a	\0

- 字符串的有效长度比字符数组的长度更重要。
- 为了测定字符串的实际长度，C语言规定了字符串结束标志，以字符'\0'作为结束标志。

"C program"

字符串是存放在一维数组中的，占10个字节，字符占9个字节，最后一个字节'\0'是由系统自动加上的



- C系统在用字符数组存储字符串常量时，会自动加一个'\0'作为结束符。
- 在定义字符数组时应估计实际字符串长度，保证数组长度始终大于字符串实际长度。
- 如果在一个字符数组中先后存放多个不同长度的字符串，应使数组长度大于最长的字符串的长度。

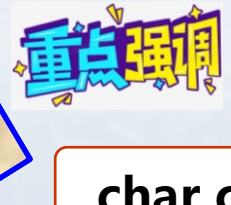
字符串和字符串结束标志

printf("How do you do?\n");

在向内存中存储时，系统自动在最后一个字符'\n'的后面加了一个'\0'，作为字符串结束标志。在执行printf函数时，每输出一个字符检查一次，看下一个字符是否为'\0'，遇'\0'就停止输出。

```
char c[]={"I  am  happy"};        或
char c[]="I am happy";
```

用一个字符串(注意字符串的两端是用双引号而不是单引号括起来的)作为字符数组的初值。



★ 数组c的长度不是10，而是11。因为字符串常量的最后由系统加上一个'\0'。

```
char c[]={'I',' ','a','m',' ','h','a','p','p','y','\0'};  ≠  char c[]={'I',' ','a','m',' ','h','a','p','p','y'};
```

```
char c[10]={"China"};
```

C	h	i	n	a	\0	\0	\0	\0	\0

数组c的前5个元素为: 'C','h','i','n','a',第6个元素为'\0'，后4个元素也自动设定为空字符。

单选题 100分

设置

有语句`char str[]="program";`

执行`printf("%d",strlen(str));` 则输出结果为(　　)

A 5

B 6

C 7

D 8

提交

字符数组的输入输出

1. 逐个字符输入输出。用格式符%c输入或输出一个字符。
2. 将整个字符串一次输入或输出。用%s格式符，意思是对字符串(string)的输入输出。

```
#include <stdio.h>
int main(void)
{
  char c[15]={'I',' ','a','m',' ','a',' ','s','t','u','d','e','n','t','.' };
  int i;
  for(i=0;i<15;i++)
    printf("%c",c[i]);
  printf("\n");
  return 0;
}
```

```
#include <stdio.h>
int main(void)
{
   char c[]="China";
   printf("%s\n",c);
   return 0;
}
```

敲黑板！

- 输出的字符中不包括结束符'\0'。
- 用%s格式符输出字符串时，printf函数中的输出项是字符数组名，而不是数组元素名。
- 如果数组长度大于字符串的实际长度，也只输出到遇'\0'结束。
- 如果一个字符数组中包含一个以上'\0'，则遇第一个'\0'时输出就结束。

字符数组的输入输出


```
char c[6];
scanf("%s",c);
```

从键盘输入: (遇空格或回车结束)

China↙

系统会自动在China后面加一个'\0'结束符。

c	h	i	n	a	\0

如果利用一个scanf函数输入多个字符串，则应在输入时以空格分隔。

```
char str1[5],str2[5],str3[5];
scanf("%s%s%s",str1,str2,str3);
```

从键盘输入:

How are you? ↙

由于有空格字符分隔，作为3个字符串输入。

str1:	H	o	w	\0	\0
str2:	a	r	e	\0	\0
str3:	y	o	u	?	\0

```
char str[13];
scanf("%s",str);
```

从键盘输入:

How are you? ↙

H	o	w	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0

由于系统把空格字符作为输入的字符串之间的分隔符，因此只将空格前的字符"How"送到str中。

字符数组的输入输出



➔ scanf函数中的输入项如果是字符数组名，不要再加地址符&。C语言中数组名代表该数组第一个元素的地址（即：数组的起始地址）。



```
scanf("%s", &str);                    //str前面不应加&
```

➔ 若数组占6个字节。数组名c代表地址2000。输出数组第一个元素的地址语句为：

```
printf("%o",c);          //用八进制形式输出数组c的起始地址
```

	c数组
2000	C
2001	h
2002	i
2003	n
2004	a
2005	\0

➔ 计算机实际执行： 按字符数组名c找到其数组第一个元素的地址，然后逐个输出其中的字符，直到遇'\0'为止。

```
printf("%s",c);          //用八进制形式输出数组c的起始地址
```

例6-8

字符串处理函数

- 为方便字符串的处理，C语言编译系统中提供了大量有关字符串处理的库函数，这些库函数为字符串处理提供了方便。
- 字符串处理函数必有包含头文件<string.h>

输入字符串的函数

gets(字符数组)

```
gets(str);          //str是已定义的字符数组
```

- 作用：从终端输入一个字符串到字符数组，并且得到一个函数值。该函数值是字符数组的起始地址。

如果从键盘输入：

Computer↙

将输入的字符串"Computer"送给字符数组str(强调：送给数组的共有9个字符，而不是8个字符)，返回的函数值是字符数组str的第一个元素的地址。

输入字符串的函数

fgets(字符数组,长度,stdin);

```
char s[15];
n = 15;
fgets(s, n, stdin);
```

- **作用：从指定的流中读取字符串。**
  - stdin是C语言中标准输入流，一般用于获取键盘到缓冲区的数据。由于stdin已经在stdio.h中定义，故不需单独定义。
  - scanf、getchar都是从stdin流中获取指定的数据，其中，scanf根据第一个参数指定的格式符将数据读入参数列表中对应参数指定的内存位置中，getchar是从stdin流中读取一个字符并返回。

- s字符数组名，输入的字符串存放在s数组中；
- n是要读取的最大字符数(包括最后的空字符)，通常使用数组str的长度；
- stdin是指向FILE对象的指针，用于标识要从中读取字符的流。

输入字符串的函数

gets函数和fgets函数均可以读取一个字符串，区别为：

gets函数是从标准输入设备读字符串，即从stdin流中获取一个字符串；

fgets函数可以从任意指定的流中获取字符串。



gets可以无限读取，不会判断上限，以回车结束读取，应该确保缓存的空间足够大，以便在执行读操作时不发生溢出；

fgets函数当读取n-1个变量时，或者读取到换行符时，或者到达文件末尾时，它会停止，具体视情况而定。

gets函数将换行符从缓存区取出丢掉；

fgets函数将换行符从缓冲区取出来留下，用fgets函数输入字符串时，字符串末尾将携带有换行符，缓冲区清空。若缓存区字符串长度超出fgets函数指定的长度，需手动清空。

GCC编译器不能使用gets函数（有版本可以），需用fgets函数。微软的VS编译器两者皆可。

输入字符串的函数

- 为便于在鲲鹏平台上**正确读取字符串**，定义函数：

手动清空函数

```
void delFgets(char *str)
{
    int n = strlen(str) - 1;
    if (str[n] == '\n')
        str[strlen(str) - 1] = '\0'; // 将字符串中读入的换行符删除
    else
        while (getchar() != '\n') // 清空输入缓冲区中本行输入的多余字符序列
            ;
}
```

输出字符串的函数

puts(字符数组)

- 作用：将一个字符串(以'\0'结束的字符序列)输出到终端。
- 用puts函数输出的字符串中可以包含转义字符。
- 在用puts输出时将字符串结束标志'\0'转换成'\n'，即输出完字符串后换行。

```
#include <stdio.h>
int main(void)
{
    char  str[]={"China\nBeijing"};
     puts(str);
     return 0;
}
```



📖 用puts和gets函数只能输出或输入一个字符串。

puts(str1,str2); 或 gets(str1,str2);



字符串比较函数

strcmp(字符串1，字符串2)

- 作用：比较字符串1和字符串2。
- 字符串比较的规则是：将两个字符串自左至右逐个字符相比（按ASCII码值大小比较），直到出现不同的字符或遇到'\0'为止。

① 全部字符相同，则认为两个字符串相等；

② 若出现不相同的字符，则以第1对不相同的字符的比较结果为准。

```
strcmp(str1, str2);
strcmp("China", "Korea");
strcmp(str1, "Beijing");
```

- 比较的结果由函数值带回。

① 如果字符串1与字符串2相同，则函数值为0。

② 如果字符串1＞字符串2，则函数值为一个正整数。

③ 如果字符串1＜字符串2，则函数值为一个负整数。



对两个字符串比较不能直接用str1＞str2进行比较，因为str1和str2代表地址而不代表数组中全部元素，而只能用（strcmp(str1,str2)>0）实现，系统分别找到两个字符数组的第一个元素，然后顺序比较数组中各个元素的值。

字符串复制函数

strcpy(字符数组1，字符串2)

```
char str1[10], str2[]="China";
strcpy(str1, str2); 或 strcpy(str1, "China");
```

- 作用：将字符串2复制到字符数组1中去。

C	h	i	n	a	\0	\0	\0	\0	\0

- 字符数组1必须定义得足够大，以便容纳被复制的字符串2。字符数组1的长度不应小于字符串2的长度。
- 字符数组1必须写成数组名形式(表示地址)，字符串2可以是字符数组名，也可以是一个字符串常量。
- 在复制前若未对字符数组1初始化或赋值，则其各字节中的内容无法预知，复制时将字符串2和其后的'\0'一起复制到字符数组1中，取代字符数组1中前面的字符，未被取代的字符保持原有内容。
- 不能用赋值语句将一个字符串常量或字符数组直接给一个字符数组。字符数组名是一个地址常量，不能改变值，正如数值型数组名不能被赋值一样。



```
str1="China"; str1=str2;
```

- 可以用strncpy函数将字符串2中前面n个字符复制到字符数组1中去。

```
strncpy(str1, str2, 2);
```



将str2中最前面2个字符复制到str1中，取代str1中原有的最前面2个字符。但复制的字符个数n不应多于str1中原有的字符(不包括'\0')。

字符串连接函数

strcat(字符数组1，字符数组2)

- 作用：把两个字符数组中的字符串连接起来，把字符串2接到字符串1的后面，结果放在字符数组1中，函数调用后得到一个函数数值—字符数组1的地址。
- 字符数组1必须足够大，以便容纳连接后的新字符串。
- 连接前两个字符串的后面都有'\0'，连接时将字符串1后面的'\0'取消，只在新串最后保留'\0'。

```
char str1[30]={"People's Republic of "};
char str2[]={"China"};
printf("%s", strcat(str1, str2));
```

连接前 str1:	P	e	o	p	l	e	'	s		R	e	p	u	b	l	i	c		o	f		\0	\0	\0	\0	\0	\0	\0	\0	\0
str2:	C	h	i	n	a																									
连接后 str1:	P	e	o	p	l	e	'	s		R	e	p	u	b	l	i	c		o	f		C	h	i	n	a	\0	\0	\0	\0

测字符串长度的函数

strlen(字符数组)

- 作用：测试字符串长度的函数。
  函数的值为字符串中的实际长度（不包括'\0'在内）。

```
#include <stdio.h>
#include <string.h>
int main(void)
{
    char str[10]="China";
    printf("%d,%d\n",strlen(str),strlen("China"));
}
```

```
[root@swjtu-kp chpt-6]# gcc test.cpp -o test
[root@swjtu-kp chpt-6]# ./test
5,5
```

转换为大小写的函数

strlwr(字符串)

- 将字符串中大写字母换成小写字母。

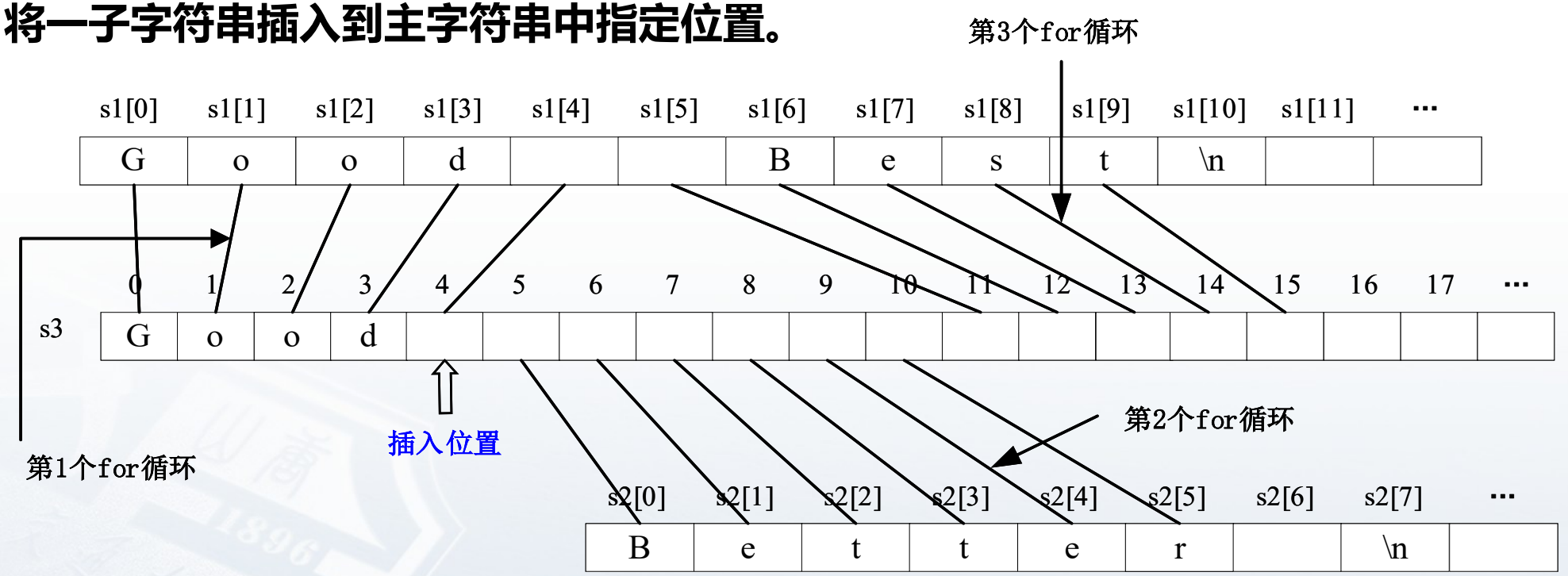
strupr(字符串)

- 将字符串中小写字母换成大写字母。



- 8种字符串处理函数，属于**库函数**。库函数并非C语言本身的组成部分，而是C语言编译系统为方便用户使用而提供的**公共函数**。不同的编译系统提供的**函数数量**和**函数名**、**函数功能**都不尽相同，使用时要小心，必要时查阅**库函数手册**。
- 在使用字符串处理函数时，应当在程序文件的开头用`#include<string.h>`把`string.h`文件**包含到本文件**中。

例6-9 字符串的插入：将一子字符串插入到主字符串中指定位置。



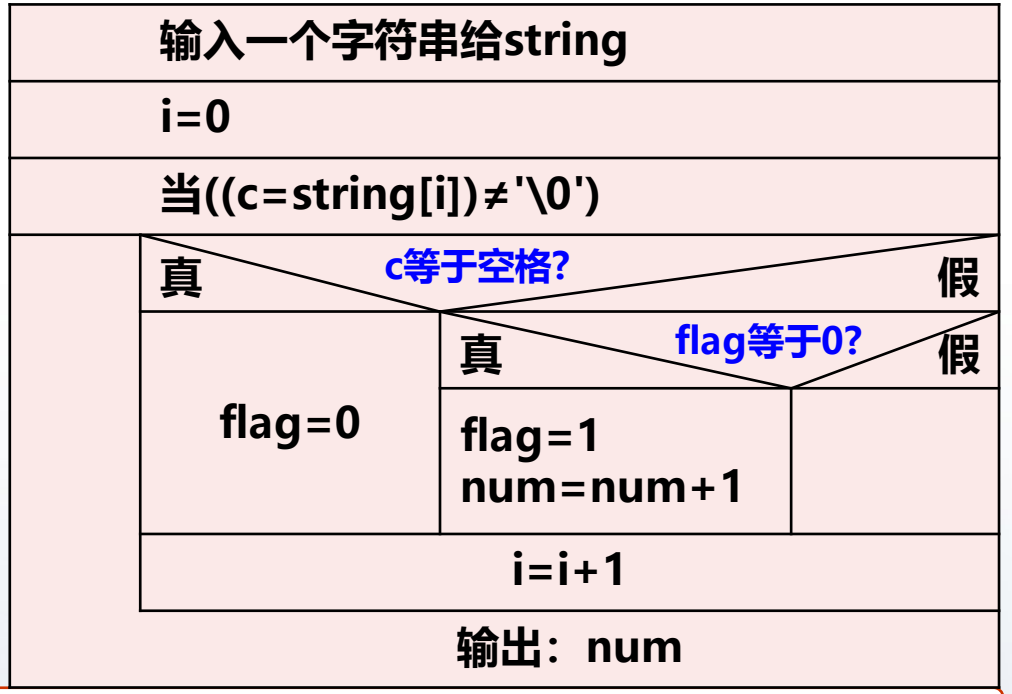

```
for (i = 0; i < n; i++) /*将主串插入位置前的字符拷贝给结果串s3*/
    s3[i] = s1[i];
for (j = 0; s2[j] != '\0'; j++) /*接着将子串所有字符拷贝到结果串尾部*/
    s3[i + j] = s2[j];
for (k = n; s1[k] != '\0'; k++) /*继续将主串余下字符拷贝到结果串*/
    s3[j + k] = s1[k];
```

例6-10

字符数组应用举例

例6-11 输入一行字符，统计其中有多少个单词，单词之间用空格分隔开。

- string：用于存放字符串。
- i：计数器，用于遍历字符串中的每个字符。
- flag：用于判断是否开始了一个新单词的标志。若flag=0表示未出现新单词，如出现了新单词，就把flag置成1。
- num：用于统计单词数。




```
#include <stdio.h>
int main(void)
{
    char string[81];
    int i,num=0,flag=0;
    char c;
    printf("请输入一个英文句子:");
    fgets(string, 81, stdin); //输入一个字符串
    for(i=0;(c=string[i])!='\0';i++) //字符不是'\0'就循环
      if(c==' ') flag=0;    //若是空格字符，使flag置0
      else if(flag==0)      //如果不是空格字符且flag原值为0
      {  flag=1;            //使flag置1
         num++;             //num累加1，表示增加一个单词
      }
    printf("一共有%d个单词.\n", num);
    return 0;
}
```

数组与函数

一维数组元素作为函数参数

一维数组名作为函数参数

函数形参和实参

分配
独立内存单元

共享
实参数组内存单元

有
确定值

形式参数	实际参数
变量	常量、变量、表达式、数组元素
数组	数组

数组元素作为函数实参

数组元素可以用作函数实参，但是不能用作形参。因为形参是在函数被调用时临时分配存储单元的，不可能为一个数组元素单独分配存储单元（数组是一个整体，在内存中占连续的一段存储单元）。在用数组元素作函数实参时，把实参的值传给形参，是值传递方式。数据传递的方向是从实参传到形参，单向传递。

形参	变量定义
实参	数组元素

将数组元素看作普通变量

数组元素作函数实参



用数组元素作实参时，向形参变量传递的是数组元素的值

示例 输入N个数，要求输出其中值最大的元素和该数是第几个数



```
#include <stdio.h>
#define N 10
int main(void)
{    int max(int x,int y);              //函数声明
     int a[N],m,n,i;
     printf("输入 %d 个整数: ",N);
     for(i=0;i<N;i++)                   //输入N个数给a[0]~a[N-1]
          scanf("%d",&a[i]);
     printf("\n");
     for(i=1,m=a[0],n=0;i<N;i++)
     {    if(max(m,a[i])>m)    //若max函数返回的值大于m
               {    m=max(m,a[i]); //max函数返回的值取代m原值
                    n=i;      //把此数组元素的序号记下来，放在n中
               }
     }
     printf("The largest number is %d\nit is the %dth number.\n",m,n+1);
}
```

```
int max(int x,int y)          //定义max函数
{
     return  x>y?x:y;         //返回x和y中的大者
}
```

例6-12



📖变量m用来存放当前已比较过的各数中的最大者。开始时设m的值为a[0]，再依次将m与a[i]比，如果a[i]大于m，就以a[i]的值取代m的原值。下一次以m的新值与下一个a[i]比较。经过N-1轮循环的比较，m最后的值就是N个数的最大数。

📖当每次出现以max(m,a[i])的值取代m的原值时，就把i的值保存在变量n中。n最后的值就是最大数的序号(注意序号从0开始)，若要输出最大数是N个数中第几个数，应为n+1。因为数组元素序号从0开始。

一维数组名作函数参数

- 数组参数的传递（实参为数组名，形参为数组定义）
  - 数组作为参数时，会将实参数组的起始地址传递给形参。即实参和形参表示同一数组，因此，形参数组内容的修改，也是对实参数组内容的修改。
  - 实参数组和形参数组的类型必须一致。维数和大小也最好一致。
  - 形参数组第一维的大小可以省略，但不能省略其他维的大小。

形参	数组定义
实参	数组名

发生函数调用：

形参数组就共享实参数组

一维数组经典程序

- 插入算法
- 删除算法
- 求平均值算法
- 排序算法（比较互换法、选择法、冒泡法）
- 查找算法（线性查找和折半查找）

求平均值

例6-13 有一个一维数组score，内放N个学生成绩，求平均成绩。

```
#include <stdio.h>
#define N 10
int main(void)
{    float average(float array[],int n);//函数声明
     float score[N],aver;
     int i;
     printf("请输入%d个学生成绩:\n",N);
     for(i=0;i<N;i++)
          scanf("%f",&score[i]);
     aver=average(score,N);        //调用average函数
     printf("平均成绩为: %.2f\n",aver);
     return 0;
}
float average(float array[],int n) //定义average函数
{    int i;
     float aver,sum=array[0];
     for(i=1;i<n;i++)
          sum=sum+array[i];        //累加学生成绩
     aver=sum/n;
     return aver;
}
```

调用average函数后：形参数组array就是实参数组score的别名



- 用数组名作函数参数，应该在主调函数和被调用函数分别定义数组。
- 实参数组与形参数组类型必须一致。
- 在定义average函数时，声明形参数组的大小为空，因为指定其大小是不起任何作用的，因为C语言编译系统并不检查形参数组大小，只是将实参数组的首元素的地址传给形参数组名。

- 用数组名作函数实参时，向形参（数组定义或指针变量）传递的是数组首元素的地址。
- 形参数组可以不指定大小，在定义数组时在数组名后面跟一个空的方括号。

强调 Emphasize

求平均值

示例 有两个班级，分别有5和10名学生，调用average函数，分别求这两个班的学生的平均成绩。

```
#include <stdio.h>
int main(void)
{    float average(float array[],int n);
     float score1[5]={98.5,97,91.5,60,55};//定义长度为5的数组
     float score2[10]={88,76,90,70,85,66,97,86,92,83};
     //定义长度为10的数组
     printf("The average of class A is %6.2f\n",average(score1,5));
     //用数组名score1和5作实参
     printf("The average of class B is %6.2f\n",average(score2,10));
     //用数组名score2和10作实参
     return 0;     }
```

```
float average(float array[],int n) //定义average函数，未指定形参数组长度
{    int i;
     float aver,sum=array[0];
     for(i=1;i<n;i++)
          sum=sum+array[i];     //累加n个学生成绩
     aver=sum/n;
     return aver;
}
```

重点强调

用数组名作函数实参时，不是把数组元素的值传递给形参，而是把实参数组的首元素的地址传递给形参数组，这样两个数组就共占同一段内存单元。

score1/score2	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
起始地址1000（假定）	88	76	90	70	85	66	97	86	92	83
array	[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

比较互换法排序

例6-14a 用比较互换法实现N个整数的降序排列。

第0步

排好a[0]：将a[0]依次与a[1]，a[2]，……，a[N-1]比较，在比较过程中，如果a[0]小于与之相比较的任意一个数组元素，就将a[0]与对应的数组元素进行交换。这样，a[0]在不断增大，比较完后，a[0]中的值就是N个数中的最大者。一般把这样的一步操作称为一趟(或一轮)排序。

第1步

排好a[1]：将a[1]依次与a[2],a[3],…,a[N-1]比较，必要时交换，这样a[1]的值就是a[1]，a[2]，……，a[N-1]中的最大者。

…

……

第N-2步

将a[N-2]与a[N-1]比较，并将较大的数放在a[N-2]中。

	a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]
排序前：	60	50	100	20	40	90	10	30	70	80
首趟排序后：	100	50	60	20	40	90	10	30	70	80
排序完成：	100	90	80	70	60	50	40	30	20	10

例6-14a 例6-14b

选择法排序

例6-14c 用选择排序法实现N个整数的升序排列。

- 选择排序法是对比较互换法进行优化，找位置再交换
- 思路：所谓选择法就是先将N(N=5)个数中最小的数与a[0]对换；再将a[1]~a[N-1]中最小的数与a[1]对换……每比较一轮，找出一个未经排序的数中最小的一个。共比较N-1轮。

a[0]	a[1]	a[2]	a[3]	a[4]	
3	6	1	9	4	未排序时的情况
1	6	3	9	4	将5个数中最小的数1与a[0]对换
1	3	6	9	4	将余下的后面4个数中最小的数3与a[1]对换
1	3	4	9	6	将余下的3个数中最小的数4与a[2]对换
1	3	4	6	9	将余下的2个数中最小的数6与a[3]对换，至此完成排序



调用select函数后

形参数组b就是实参数组a的别名

```
#include <stdio.h>
#define N 10
int main(void)
{   void select(int b[],int n);
    int a[N],i;
    printf("输入数据: ");
    for(i=0;i<N;i++)
        scanf("%d",&a[i]);
    select(a,N);     //调用select函数,a为数组名,大小为N
    printf("升序排序结果为: ");
    for(i=0;i<N;i++)
        printf("%d ",a[i]);
    printf("\n");
    return 0;
}
```

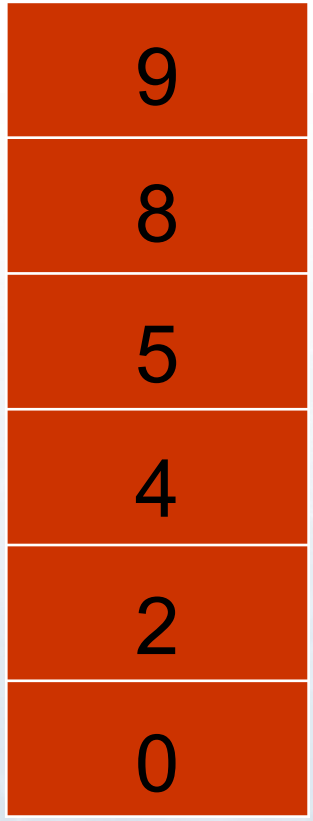
```
void select(int b[],int n)
{
    int i,j,min,t;
    for(i=0;i<n-1;i++)
    {   min=i;  //min保存当前排序数中最小数下标
        for(j=i+1;j<n;j++)
            if(b[j]<b[min])
                min=j;
        if(min!=i)
        {  t=b[min]; b[min]=b[i]; b[i]=t; }
    }
}
```



执行函数调用语句select(a,N);之前和之后，a数组中各元素的值是不同的。排序前是无序的，执行select(a,N);后，a数组已经排好序。调用select函数，参数组b成为实参数组a的别名，形参数组改变即是实参数组改变。

冒泡法排序

例6-14d 用**冒泡排序法**实现N个整数的**升序**排列：按**相邻原则两两比较**待排序序列中的元素，并交换不满足顺序要求的各对元素



第一趟



冒泡法排序

第1步

- 第一趟冒泡排序：将第1个元素与第2个元素进行比较，若为逆序，则将两元素交换。比较第2个与第3个元素，依次类推，直到第N-1个和第N个元素进行比较和交换。经过第一趟冒泡排序，最大元素被交换到第N个位置。（大数沉底，小数上浮）

第1步

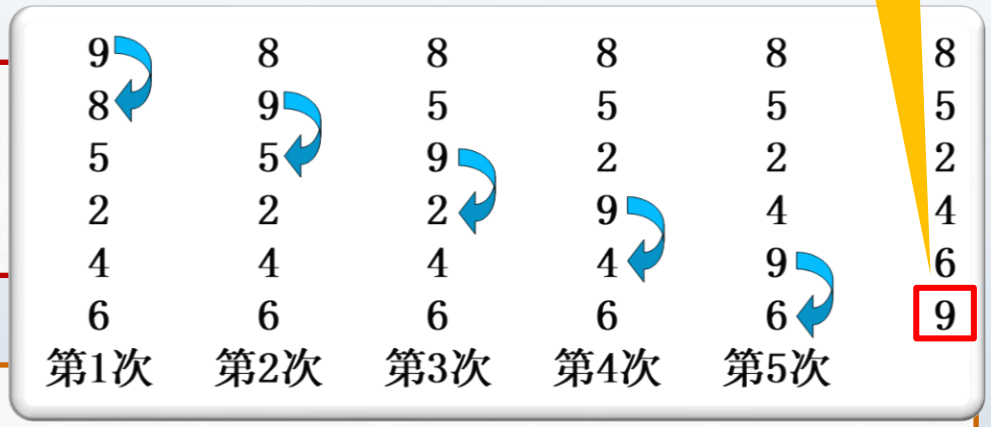
- 对前N-1个元素进行第二趟起泡排序，将其中的最大元素交换到第N-1个位置。

…

- ……

第N-2步

- 直到所有的比较次数结束，或在某一趟排序比较中未发生任何交换时，排序结束。

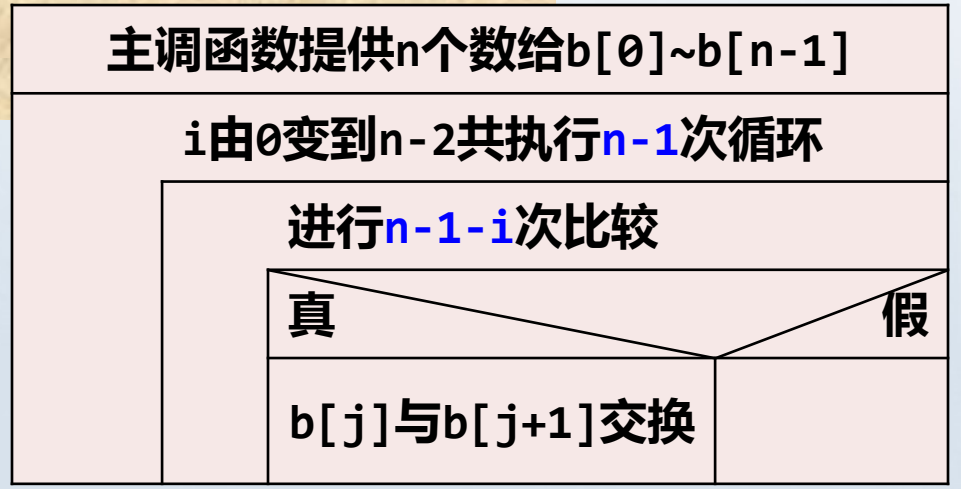


例6-14d

冒泡法排序

例6-14d 用起泡法(冒泡法)对N(N=10)个整数由小到大的顺序排列。

```
void bubble(int b[], int n) //bubble完成n个整数的升序排列
{
    int i, j, t;
    for (i = 0; i < n - 1; i++)      //外层循环：进行n-1次循环，实现n-1趟比较
        for (j = 0; j < n - 1 - i; j++) //内层循环：在每一趟中进行n-1-i次比较
            if (b[j] > b[j + 1])     //相邻两个数比较：升序排列，必须前小后大
            {
                t = b[j];
                b[j] = b[j + 1];
                b[j + 1] = t;
            }
}
```

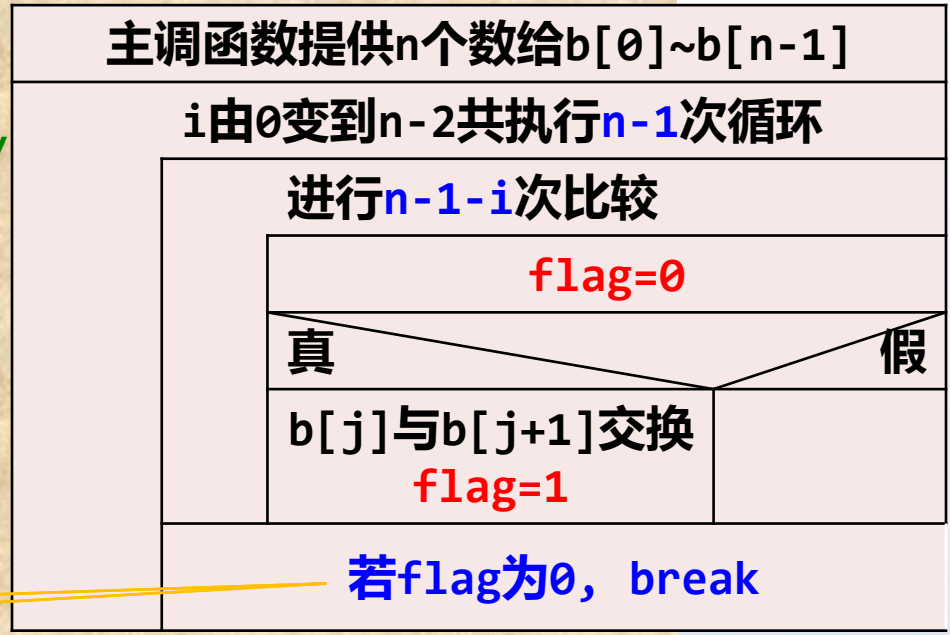


冒泡法排序

例6-14e 优化的冒泡排序法实现N个整数的升序排列

优化算法，以减少循环次数。

```
void good_bubble(int b[], int n) /*good_bubble完成n个整数的升序排列*/
{
    int i, j, flag, t;
    for (i = 0; i < n - 1; i++) /*i为排序的趟数*/
    {
        flag = 0; /*设置标志变量，用于判断本趟是否发生数据交换*/
        for (j = 0; j < n - 1 - i; j++)
            if (b[j] > b[j + 1]) /*升序排列，必须前小后大*/
            {
                t = b[j];
                b[j] = b[j + 1];
                b[j + 1] = t;
                flag = 1; /*有数据交换发生，即将flag置为1*/
            }
        if (!flag)
            break; /*若本趟没有发生过数据交换，则待排序数据已排好序，提前结束排序*/
    }
}
```



查找问题

例6-15 数组元素的查找：在数组中搜索一个特定元素的处理过程

- 简单的线性查找
- 效率更高的折半查找(但算法较复杂)：对于一个有序排列的数组，如果查找某一个元素，通常用折半查找法(二分查找)。

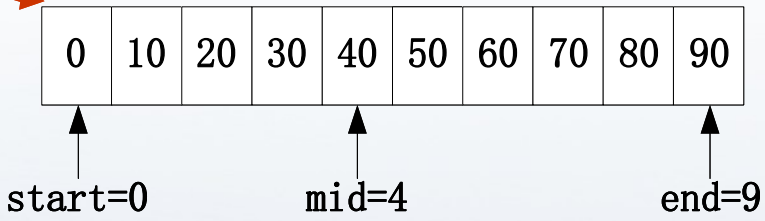
例6-15a

例6-15b

例6-15c

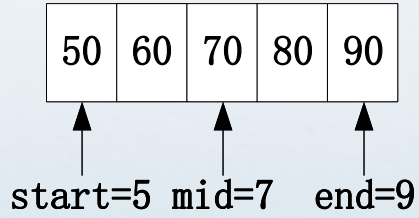
第1步

将数组中间位置记录，比较查找关键字与中间位置的元素，若两者相等，查找成功，否则从中间位置将数组一分为二。



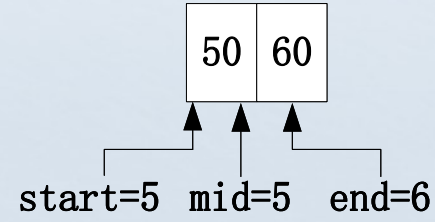
第2步

如果关键字大于中间位置元素，则在后一半数组中查找，否则在前一半数组中查找。



第3步

重复以上过程，直到找到满足条件的记录，使查找成功，若直到子数组不存在，没有找到要查找的元素，则元素不存在。



二维数组

怎样定义二维数组

怎样使用二维数组

定义和引用二维数组

小例子

有3个小分队，每队有6名队员，要把这些队员的工资用数组保存起来以备查。

	队员1	队员2	队员3	队员4	队员5	队员6
第1分队	2456	1847	1243	1600	2346	2757
第2分队	3045	2018	1725	2020	2458	1436
第3分队	1427	1175	1046	1976	1477	2018

- 如果建立一个数组pay，必须是二维的，第一维用来表示第几分队，第二维用来表示第几个队员。例如用$pay_{2,3}$表示2分队第3名队员的工资，值是1725。
- 二维数组常称为矩阵(matrix)。把二维数组写成行(row)和列(column)的排列形式，有助于形象化地理解二维数组的逻辑结构。

定义二维数组

类型说明符　数组名[常量表达式][常量表达式]

```
float pay[3][6];
```

- float型二维数组
- 数组名为pay
- 数组第二维有6个元素
- 数组第一维有3个元素

```
float a[3][4], b[5][10];
//定义a为3×4(3行4列)的数组，b为5×10(5行10列)的数组
```

```
float a[3, 4], b[5, 10];    //在一对方括号内不能写两个下标
```

二维数组可被看作一种特殊的一维数组：其元素又是一个一维数组。

```
float a[3][4];
```

可以把a看作一个一维数组，有3个元素：a[0]，a[1]，a[2]，每个元素又是一个包含4个元素的一维数组：

- a[0] — a[0][0]　a[0][1]　a[0][2]　a[0][3]
- a[1] — a[1][0]　a[1][1]　a[1][2]　a[1][3]
- a[2] — a[2][0]　a[2][1]　a[2][2]　a[2][3]

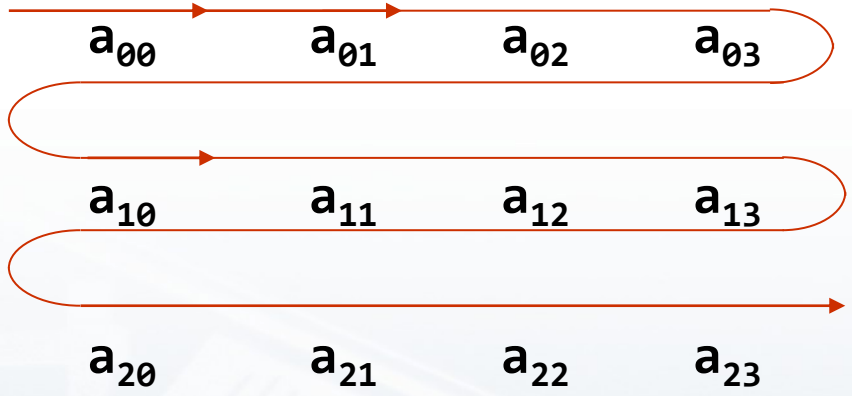
	列0	列1	列2	列3
行0	a[0][0]	a[0][1]	a[0][2]	a[0][3]
行1	a[1][0]	a[1][1]	a[1][2]	a[1][3]
行2	a[2][0]	a[2][1]	a[2][2]	a[2][3]

列下标
行下标
数组名

二维数组的存储

C语言中，二维数组中元素排列的顺序是**按行存放**的。

```
float a[3][4]
```



2000	a[0][0]	第0行元素
2004	a[0][1]	
2008	a[0][2]	
2012	a[0][3]	
2016	a[1][0]	第1行元素
2020	a[1][1]	
2024	a[1][2]	
2028	a[1][3]	
2032	a[2][0]	第2行元素
2036	a[2][1]	
2040	a[2][2]	
2044	a[2][3]	



➔ 用**矩阵形式**（如3行4列形式）表示二维数组，是**逻辑**上的概念，能形象表示出**行列关系**。而在**内存**中，各元素是**连续存放**的，不是二维的，是**线性**的。

```
float a[2][3][4];//定义三维数组a，它有2页，3行，4列
```



多维数组元素在内存中的排列顺序为：第1维的下标变化最慢，最右边的下标变化最快。

`float a[2][3][4];`在内存中的排列顺序为：

```
a[0][0][0] → a[0][0][1] → a[0][0][2] → a[0][0][3] → a[0][1][0] → a[0][1][1] →
a[0][1][2] → a[0][1][3] → a[0][2][0] → a[0][2][1] → a[0][2][2] → a[0][2][3] →
a[1][0][0] → a[1][0][1] → a[1][0][2] → a[1][0][3] → a[1][1][0] → a[1][1][1] →
a[1][1][2] → a[1][1][3] → a[1][2][0] → a[1][2][1] → a[1][2][2] → a[1][2][3]
```

引用二维数组元素

数组名[下标][下标]

- 下标可以是整型常量或整型表达式。
- 数组元素可以出现在表达式中，也可以被赋值

```
b[1][2]=a[2][3]/2;
```



➔ 在引用数组元素时，下标值应在已定义的数组大小的范围内。

```
int a[3][4];   //定义a为3×4的二维数组

a[3][4]=3;  //不存在a[3][4]元素
//数组a可用的行下标的范围为0~2，列下标的范围为0~3
```



➔ 严格区分

- 定义数组时用的a[3][4]：定义数组的维数和各维的大小）
- 引用元素时的a[3][4]：3和4是数组元素的下标值，a[3][4]代表行序号为3、列序号为4的元素行序号和列序号均从0开始）。

单选题 100分

设置

若定义了一个4行3列的数组，则第10个元素是。（　　）

A　a[2][2]

B　a[2][3]

C　a[3][1]

D　a[3][0]

提交

二维数组的初始化

可以用初始化列表对二维数组初始化。

1. 分行给二维数组赋初值。（最清楚直观）

```
int a[3][4]={{1,2,3,4},{5,6,7,8},{9,10,11,12}};
```

2. 可以将所有数据写在一个花括号内，按数组元素在内存中的排列顺序对各元素赋初值。

```
int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

3. 可以对部分元素赋初值。

```
int a[3][4]={{1},{5},{9}};          ①
int a[3][4]={{1},{0,6},{0,0,11}};   ②
int a[3][4]={{1},{5,6}};            ③
int a[3][4]={{1},{},{9}};           ④
```

①

1	0	0	0
5	0	0	0
9	0	0	0

②

1	0	0	0
0	6	0	0
0	0	11	0

③

1	0	0	0
5	6	0	0
0	0	0	0

④

1	0	0	0
0	0	0	0
9	0	0	0

4. 如果对全部元素都赋初值(即提供全部初始数据)，则定义数组时对第1维的长度可以不指定，但第2维的长度不能省。

```
int a[3][4]={1,2,3,4,5,6,7,8,9,10,11,12};  ≡  int a[][4]={1,2,3,4,5,6,7,8,9,10,11,12};
```

5. 在定义时也可以只对部分元素赋初值而省略第1维的长度，但应分行赋初值。

```
int a[][4]={{0,0,3},{},{0,10}};
```

二维数组的初始化_总结

二维数组的方式有四种

第1种

- 按行给二维数组赋初值
- `int a[2][3] = {{1,2,3},{4,5,6}};`

第2种

- 将所有的数组元素按行顺序写在一个大括号内
- `int a[2][3] = {1,2,3,4,5,6};`

第3种

- 对部分数组元素赋初值
- `int b[3][4] = {{1},{4,3},{2,1,2}};`

第4种

- 对全部数组元素置初值，则第一个下标可省略
- `int a[ ][3] = {1,2,3,4,5,6};`

敲黑板！

若定义静态二维数组时，则数组的**所有元素将自动获取初始值0**。

单选题 100分

设置

对语句：`int array[3][4]={0,1};` 描述正确的是（　　）

A　只有元素array[0][0] 可得到初值 0

B　只有元素array[0][0] 可得到初值 0及元素array[0][1] 可得到初值1

C　此说明语句不正确

D　数组array中各元素都可得到初值

提交

单选题 100分

设置

下面初始化不正确的是（ ）

A `int a[2][3]={1,2,3,4,5,6};`

B `double a[3][3]={1,2,3,4,5,6,7,8,9,10};`

C `double a[3][2]={{1.5,2.0},{3.5},{5,6}};`

D `int a[][2]={7,8,9,10};`

提交

单选题 100分

设置

设有int a[3][3]={{1},{2,3},{4,5,6}}; 则数组元素a[1][1]的值为（　　）

A　0

B　2

C　3

D　5

提交

脚下留心

☎**数组的非法操作** 对于**基本数据类型**，**相同类型的变量**可以进行**加减、比较运算**，但对于数组，**即使是**相同类型、相同大小的数组，有些操作也是**非法的**，具体为：

①用一个已经初始化的数组对另一个数组**赋值** ✘

②对数组进行**整体输入输出** ✘

③数组与数组**进行比较** ✘

④数组与数组**进行运算** ✘



二维数组数据的输入/赋值

使用*双重循环遍历*二维数组中的每个元素：输入或赋值

$$\begin{matrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \end{matrix}$$

```
int a[M][N];
for(i=0;i<M;i++)
for(j=0;j<N;j++)
scanf("%d",&a[i][j];
```

输入

```
int a[M][N],k=1;
双层遍历
    a[i][j]=k++;
```

赋值

```
for(i=0;i<M;i++)
   for(j=0;j<N;j++)
```

```
1  2  3  4
5  6  7  8
9 10 11 12
```

```
for(j=0;j<N;j++)
   for(i=0;i<M;i++)
```

```
1  4  7 10
2  5  8 11
3  6  9 12
```

```
for(i=M-1;i>=0;i--)
   for(j=0;j<N;j++)
```

```
9 10 11 12
5  6  7  8
1  2  3  4
```

```
for(i=M-1;i>=0;i--)
   for(j=N-1;j>=0;j--)
```

```
12 11 10 9
8  7  6  5
4  3  2  1
```

```
for(j=N-1;j>=0;j--)
   for(i=M-1;i>=0;i--)
```

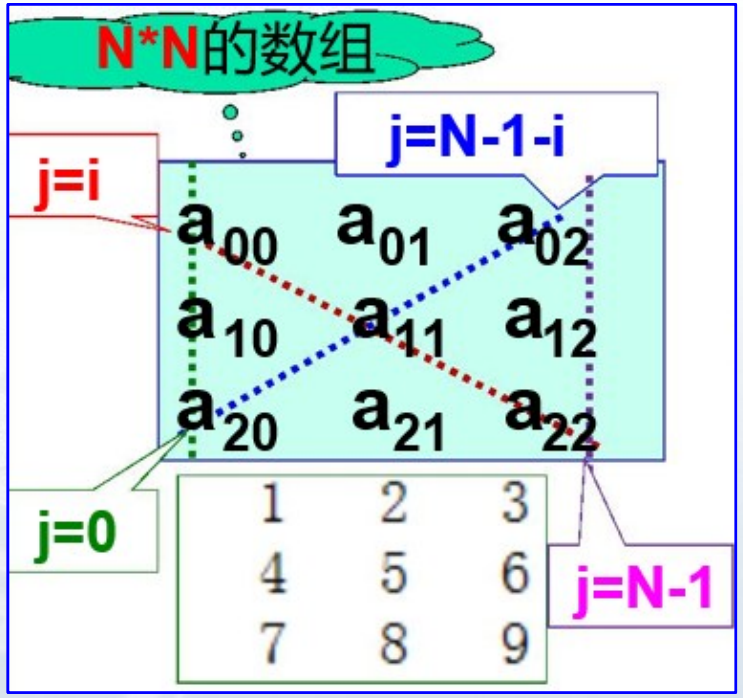
```
12 9  6  3
11 8  5  2
10 7  4  1
```

二维数组数据的输出

使用*双重循环遍历*二维数组中的每个元素：输出

$$\begin{matrix} a_{00} & a_{01} & a_{02} & a_{03} \\ a_{10} & a_{11} & a_{12} & a_{13} \\ a_{20} & a_{21} & a_{22} & a_{23} \end{matrix}$$

```
//顺序输出
for(i=0;i<M;i++)
{ for(j=0;j<N;j++)
      printf("%d ",a[i][j]);
  printf("\n");
}
```




```
//输出左下角数据
for(i=0;i<N;i++)
{  for(j=0;j<=i;j++)
       printf("%4d",a[i][j]);
   printf("\n");  }
```

```
1
4   5
7   8   9
```

```
//输出右上角数据
for(i=0;i<N;i++)
{  for(j=i;j<N;j++)
       printf("%4d",a[i][j]);
   printf("\n");  }
```

```
1   2   3
5   6
9
```

```
//输出右下角数据
for(i=0;i<N;i++)
{  for(j=N-1-i;j<N;j++)
       printf("%4d",a[i][j]);
   printf("\n");  }
```

```
3
5   6
7   8   9
```

二维数组数据的操作

西南交通大学 Southwest Jiaotong University

使用*双重循环遍历*每个元素：计算（累和、均值、查找、最大最小等）

```
for(i=0;i<M;i++)
    for(j=0;j<N;j++)
        计算功能
```

```
s=s+a[i][j];
循环外：m=x/(N*M);
```

求累和

求平均

```
if(i==j) s+=a[i][j];
```

主对角线元素和

```
if( i+j==N-1) s+=a[i][j];
```

次对角线元素和

```
if(a[i][j]==x) c++
```

顺序查找统计

```
m=a[0][0];
for(i=0;i<M;i++)
{ for(j=0;j<N;j++)
      if(a[i][j]>m)
      {   m=a[i][j];
          p=i; q=j;   }
}
```

找出最大值及位置

二维数组程序举例

例6-16 将一个二维数组行和列的元素互换(转置)，存到另一个二维数组中。

$$a=\begin{bmatrix}1 & 2 & 3\\ 4 & 5 & 6\end{bmatrix} \Rightarrow b=\begin{bmatrix}1 & 4\\ 2 & 5\\ 3 & 6\end{bmatrix}$$

```
#include <stdio.h>
int main(void)
{
    int a[2][3]={{1,2,3},{4,5,6}},b[3][2],i,j;
    printf("array a:\n");
    for(i=0;i<=1;i++)      //处理a数组中的一行中各元素
    {
        for (j=0;j<=2;j++)//处理a数组中某一列中各元素
        {
            printf("%5d",a[i][j]); //输出a数组的一个元素
            b[j][i]=a[i][j];   //将a数组元素的值赋给b数组
                               相应元素
        }
        printf("\n");
    }
    printf("array b:\n");   //输出b数组各元素
    for(i=0;i<=2;i++)       //处理b数组中一行中各元素
    {
        for(j=0;j<=1;j++)     //处理b数组中一列中各元素
            printf("%5d",b[i][j]); //输出b数组的一个元素
        printf("\n");
    }
    return 0;
}
```



处理二维数组，双重循环遍历所有元素

二维数组程序举例

示例 有一个3×4的矩阵，要求编程序求出其中值**最大**的那个元素的值，以及其所在的**行号**和**列号**。

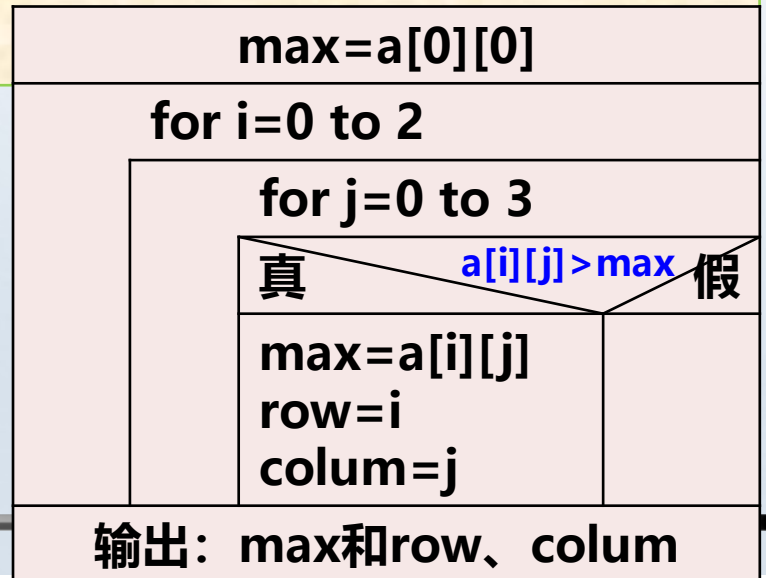
算法

找最大最小值
打擂台算法

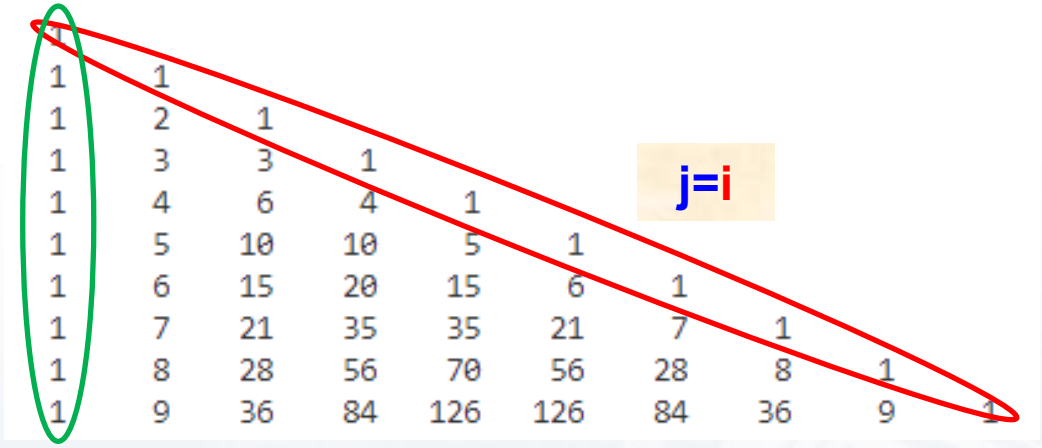
```
#include <stdio.h>
int main(void)
{    int i,j,row=0,colum=0,max;
     int a[3][4]={{1,2,3,4},{9,8,7,6},{-10,10,-5,2}};//定义数组并赋初值
     max=a[0][0];                                    //先认为a[0][0]最大
     for(i=0;i<=2;i++)
          for(j=0;j<=3;j++)
               if(a[i][j]>max)//如果某元素大于max，就取代max的原值
               {    max=a[i][j];
                    row=i;          //记下此元素的行号
                    colum=j;        //记下此元素的列号
               }
     printf("max=%d\nrow=%d\ncolum=%d\n",max,row,colum);
     return 0;
}
```

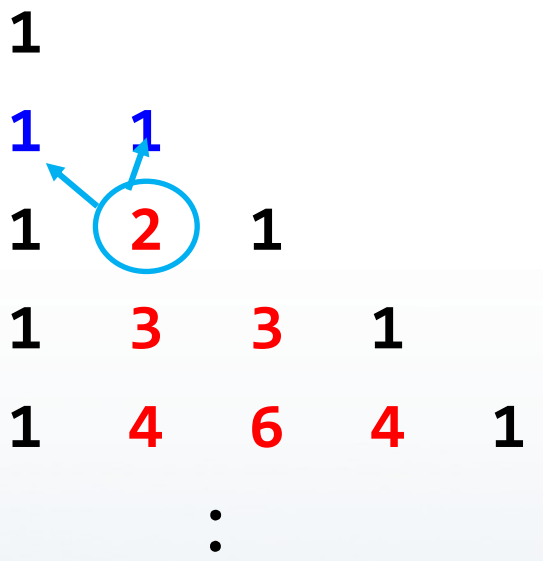
打擂台时怎样确定最后的优胜者。

- **先找出**任一人站在台上，第2人上去与之比武，胜者留在台上。
- **再上去**第3人，与台上的人（即刚才的得胜者）比武，胜者留台上，败者下台。
- **后面每人**都是与当时留在台上的人比武。直到所有人都上台比过为止，最后留在台上的就是冠军。



例6-17 输出杨辉三角形(输出N行)。





- 第1列和对角线的值是1
- 其它各元素的值：a[i][j]=a[i-1][j-1]+a[i-1][j];
  i=2~10； j=1~i-1；

```
for (i = 1; i < N; i++)  /*生成数据*/
{
    a[i][0] = 1; /*将每行0列元素赋值为*/
    a[i][i] = 1; /*将每行对角线位置列元素赋值为*/
    for (j = 1; j < i; j++)
        a[i][j] = a[i - 1][j - 1] + a[i - 1][j]; /*计算每行中间位置元素*/
}
```

例6-17

例6-17 输出杨辉三角形(右三角和等腰形式)




```
    printf("\n杨辉三角形前%d行(右对齐):\n", N);
    for (i = 0; i < N; i++)
    {
        for (j = 0; j < N - 1 - i; j++)
            printf("    ");
        for (j = 0; j <= i; j++)
            printf("%4d", a[i][j]);
        printf("\n");
    }
```

/*每行前的空格*/
/*%4d所占区域：4个空格*/
/*按行输出杨辉三角形*/

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

```
printf("\n杨辉三角形前%d行(右对齐):\n", N);
    for (i = 0; i < N; i++)
    {
        for (j = 0; j < N - 1 - i; j++)
            printf("  ");
        for (j = 0; j <= i; j++)
            printf("%4d", a[i][j]);
        printf("\n");
    }
```

/*每行前的空格*/
/*%4d所占区域的一半*/
/*按行输出杨辉三角形*/

```
1
1   1
1   2   1
1   3   3   1
1   4   6   4   1
```

```
随机生成的5行6列矩阵为：
  15   8  69  49   1  54
  12  92  26  80   8  63
   0  43  35   1   9  44
  78  23  12  81  91  88
  59  50   9  55  60  78
鞍点在2行5列，为：44
```

二维字符数组的使用方法

多个字符串可用二维字符数组

- 将一个一维字符数组末位置'\0'，则代表一个字符串；
- 将一个二维字符数组的每行末位置'\0'，则存放多个字符串。

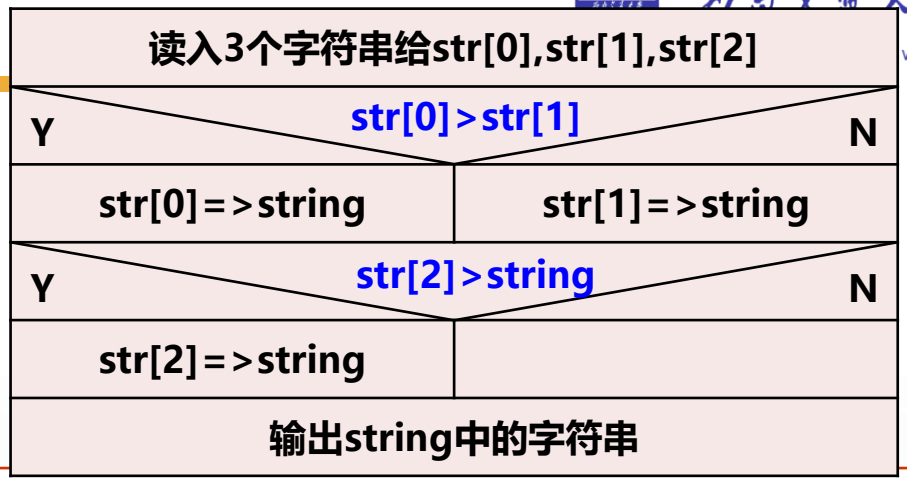
```
char cc[3][9]={"shanghai", "aba","Beijing"};
```

cc[0][ ]	s	h	a	n	g	h	a	i	\0
cc[1][ ]	a	b	a	\0					
cc[2][ ]	b	e	i	j	i	n	g	\0	

二维字符数组应用举例

示例 有3个字符串，要求找出其中最大者。

str[0]:	H	o	l	l	a	n	d	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0
str[1]:	C	h	i	n	a	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0
str[2]:	A	m	e	r	i	c	a	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0	\0




```
#include<stdio.h>
#include<string.h>
int main(void)
{
    char str[3][20];  //定义二维字符数组
    char string[20];
    //定义一维字符数组，作为交换字符串时的临时字符数组
    int i;
    for(i=0;i<3;i++)
        gets(str[i]); //读入3个字符串，分别给str[0],str[1],str[2]
    if(strcmp(str[0],str[1])>0)    //若str[0]大于str[1]
        strcpy(string,str[0]);//把str[0]的字符串赋给字符数组string
    else                           //若str[0]小于等于str[1]
        strcpy(string,str[1]);//把str[1]的字符串赋给字符数组string
    if(strcmp(str[2],string)>0)    //若str[2]大于string
        strcpy(string,str[2]);//把str[2]的字符串赋给字符数组string
    printf("\nthe largest string is:\n%s\n",string);//输出string
    return 0;
}
```

- 流程图和程序注释中的大于是指两个字符串的比较中的大于。
- str[0]，str[1]，str[2]和string是一维字符数组，其中可以存放一个字符串。
- strcpy函数在将str[0]，str[1]或str[2]复制到string时，最后都有一个'\0'。因此，最后用%s格式输出string时，遇到string中第一个'\0'即结束输出，并不是把string中的全部字符输出。

例6-18



多维数组作函数参数

- 定义函数时形参若为多维数组，形参列表中数组定义可以指定每一维的大小，也可以省略第一维的大小说明。
- 实参为数组名



```
int array[3][10]   或 int array[][10]   //二者等价
```



```
int array[][]   或 int array[3][ ] //必须指定列数
```



📖形参为二维数组时，必须指定列数(即一行中包含几个元素)，由于形参数组与实参数组类型相同，所以它们是由具有相同长度的一维数组所组成的。不能只指定第1维(行数)而省略第2维(列数)。

📖在第2维大小相同的前提下，形参数组的第1维可以与实参数组不同。记住：C编译系统不检查第1维的大小。

实参数组定义为`int score[5][10];`形参数组可定义为`int array[][10];`或`int array[8][10];`。形参数组和实参数组都是由相同类型和大小的一维数组(每行都是一个一维数组)组成。

多维数组作函数参数

示例 有一个3×4的矩阵，求所有元素中的最大值。

```
#include <stdio.h>
int main(void)
{    int max_value(int array[][4]);      //函数声明
     int a[3][4]={{1,3,5,7},{2,4,6,8},{15,17,34,12}};
     printf("Max value is %d\n",max_value(a));
     //max_value(a)为函数调用
     return 0;
}
```

实参数组

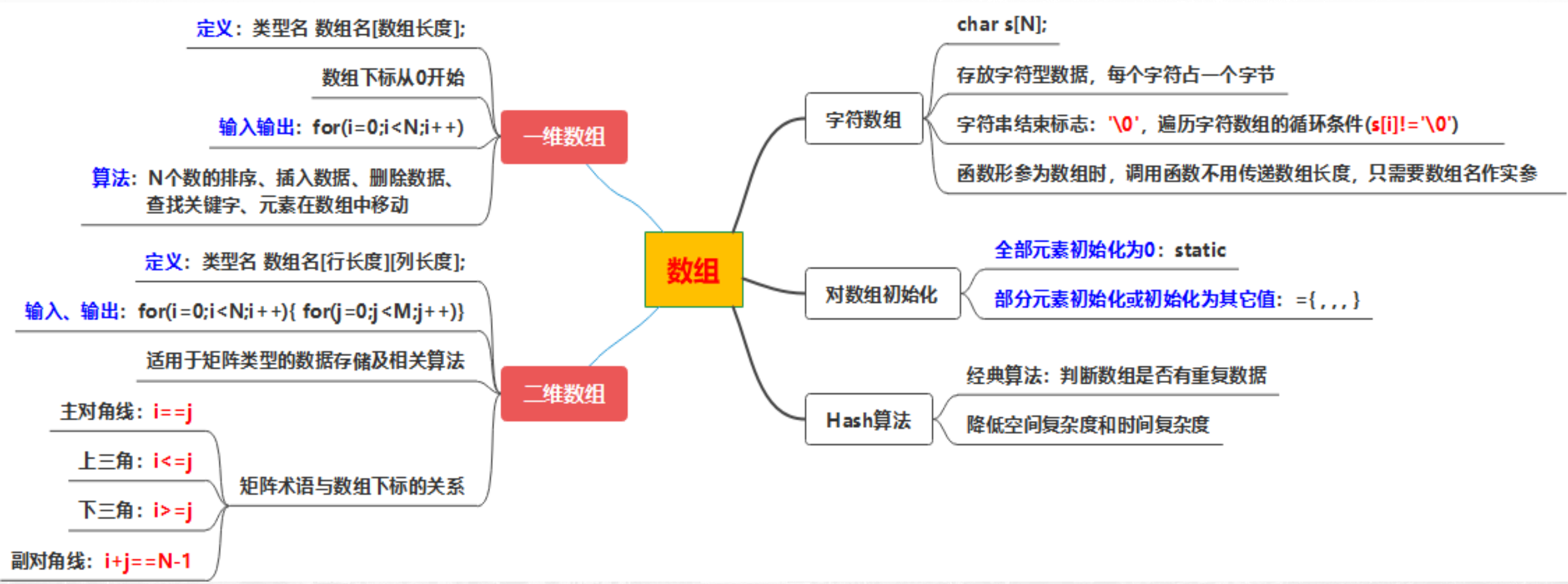
形参数组

```
int max_value(int array[][4])      //函数定义
{    int i,j,max;
     max=array[0][0];
     for(i=0;i<3;i++)
          for(j=0;j<4;j++)
               if(array[i][j]>max) max=array[i][j];
               //把大者放在max中
     return max;
}
```

例6-19　例6-20

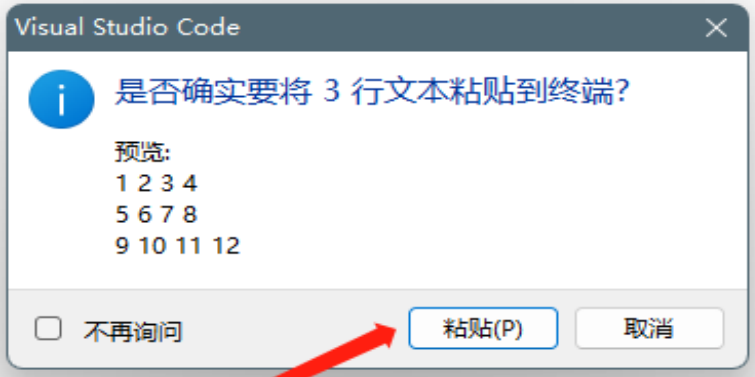
- 形参数组array第1维的大小省略，第2维大小不能省略，而且要和实参数组a的第2维的大小相同。
- 主函数调用max_value函数时，把实参二维数组a的第1行的起始地址传递给形参数组array，因此array数组第1行的起始地址与a数组的第1行的起始地址相同。
- 由于两个数组的列数相同，因此array数组第2行的起始地址与a数组的第2行的起始地址相同。
- a[i][j]与array[i][j]同占一个存储单元，具有同一个值。实际上，array[i][j]就是a[i][j]，在函数中对array[i][j]的操作就是对a[i][j]的操作。

数组思维导图


数组
一维数组
定义：类型名 数组名[数组长度];
数组下标从0开始
输入输出：for(i=0;i<N;i++)
算法：N个数的排序、插入数据、删除数据、查找关键字、元素在数组中移动
二维数组
定义：类型名 数组名[行长度][列长度];
输入、输出：for(i=0;i<N;i++){ for(j=0;j<M;j++)}
适用于矩阵类型的数据存储及相关算法
矩阵术语与数组下标的关系
主对角线：i==j
上三角：i<=j
下三角：i>=j
副对角线：i+j==N-1
字符数组
char s[N];
存放字符型数据，每个字符占一个字节
字符串结束标志：'\0'，遍历字符数组的循环条件(s[i]!='\0')
函数形参为数组时，调用函数不用传递数组长度，只需要数组名作实参
对数组初始化
全部元素初始化为0：static
部分元素初始化或初始化为其它值：={,,,}
Hash算法
经典算法：判断数组是否有重复数据
降低空间复杂度和时间复杂度

★运行程序时输入数据方法


```c
#include <stdio.h>
#include <string.h>
int main()
{
    int a[3][4], i, j;
    printf("请输入数组元素值:\n");
    for (i = 0; i < 3; i++)
        for (j = 0; j < 4; j++)
            scanf("%d", &a[i][j]);
    printf("数组元素为:\n");
    for (i = 0; i < 3; i++)
    {
        for (j = 0; j < 4; j++)
            printf("%d ", a[i][j]);
        printf("\n");
    }
}
/*
1 2 3 4
5 6 7 8
9 10 11 12
*/
//选中复制，终端中右键粘贴
```



问题 输出 终端 调试控制台 JUPYTER

```
ogxyeq.er0' '--stderr=Microsoft-MIEngine-Error-k4zispku.5dk' '--pid=Microsoft-MIEngine-Pid-cg5it5xv.40u' '--dbgExe=D:\mingw64\bin\gdb.exe' '--interpreter=mi'
请输入数组元素值:
```

鼠标右键

★字符串使用小结

- 用字符数组处理字符串：定义、输入、求长度、用字符串(能代表字符串的就是字符数组名)

  函数处理整体处理字符串或用循环处理字符串中单个字符(数组名[下标])。

  - 特别强调：字符串结束标志'\0'　　//有此标志，字符数组中存储的才认可为字符串
  - %s输入的字符串是不能含空格的!
  - 例如：char str[81];

    ① 输入

    scanf("%s",str);　　//输入的字符串遇空格或回车结束

    gets(str);　或　fgets(str,81,stdin);　//输入的字符串可含空格，以回车结束

    ② 求长度：strlen(str)

    ③ 字符串复制：strcpy(str,字符串)

    ④ 字符串连接：strcat(str,字符串)

    ⑤ 字符串比较：strcmp(字符串1,字符串2)